Auditable AI Agents

An immutable on-chain audit trail for LLM agent actions.

Master of Engineering (Computer Science and Engineering) project by **Manikandan Sankaranarayanan**, S Veersamy Chettiya College of Engineering · smak07.s@gmail.com

---

1. What it does

AI agents call tools and take actions for users. Their records are usually plain log files that anyone with server access can edit or delete without leaving a trace. This project makes the record **tamper-evident**:

1. An LLM agent runs a task. Every step (task, model response, tool call, tool result) is written to a log. Each entry contains the hash of the previous one, so entries are chained.
2. Every few entries, the entry hashes are folded into one **Merkle root** and written to a **smart contract** on a blockchain. Only these 32-byte hashes go on-chain; the full log stays in an ordinary file.
3. A **verifier** reloads the log, recomputes the hashes and compares them with the roots on the blockchain. It reports **INTACT**, or **TAMPERING DETECTED** and where.

```
task --> LLM agent + tools --> audit log (hash chain) --> log file (off-chain)
                                      |
                                      +-- every 5 entries: Merkle root --> smart contract (o
n-chain)
verifier: log file + roots on-chain --> INTACT / TAMPERING DETECTED (entry or batch)
```

What you will see when you run the demo (excerpt):

```
agent answered: The results are: 12*(3+4) = 84 ... The capital of Japan is Tokyo.
12 audit entries saved to research/experiments/agent_logs/ollama_run.jsonl
anchored batch 0: entries 0..4, root ef116e522257…, gas 93100
verify untouched log: INTACT
verify after editing entry 3: TAMPERING DETECTED
verify after consistent rewrite (only the chain can catch this): TAMPERING DETECTED
verify after restoring the original: INTACT
```

What it proves, and what it does not. It proves a log was not changed after it was anchored. It does not prove the agent's output was correct, and entries written after the latest anchor are not yet protected. See docs /methodology/evaluation-plan.md for the threat model.

---

2. What you need

Requirement	Used for	Tested with
macOS (Apple Silicon)	Everything	macOS on an Apple M5 Pro, 24 GB RAM. Other systems are untested.
Python 3	Agent, log, verifier, tests	3.14.6 (3.10 or newer is expected to work, not tested)

Requirement	Used for	Tested with
Node.js and npm	Hardhat local blockchain, contract tests	Node 26.5.0
Internet, once	pip and npm packages, the Solidity compiler (downloaded on first compile)	n/a
Ollama (optional)	Run the agent with a real local model	0.40.13
About 6 GB free disk and 8 GB free RAM (optional)	The qwen3:8b model (5.2 GB download)	n/a

You can run and test everything except the real-model demo **without Ollama**, using the built-in scripted model.

---

3. Install, step by step

Step 1. Get the code.

```
git clone https://github.com/manidaaaa/Auditable-AI-Agents.git
cd Auditable-AI-Agents
```

Step 2. Create a Python environment and install the Python packages.

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

You should see `Successfully installed ...` with `pytest`, `web3` and `anthropic` among the packages. Run `source .venv/bin/activate` again in every new terminal.

Step 3. Install the blockchain tooling.

```
npm install
```

This installs Hardhat (a local Ethereum chain and test tool) into `node_modules/`.

Step 4. Compile the smart contract.

```
npx hardhat compile
```

You should see `Compiled 1 Solidity file successfully`. The first run downloads the Solidity compiler, so it needs internet. This creates `artifacts/`, which the Python code reads.

Step 5. Check the install with the offline tests.

```
python -m pytest -q -m "not live"
```

Expected: `76 passed, 3 deselected`. Nothing else needs to be running.

Step 6 (optional). Install Ollama and a local model, to run the agent with a real model. Skip this if Ollama is already installed.

```
brew install ollama          # or download the app from https://ollama.com/download
```

Step 7 (optional). Start the Ollama server and download the model. Open a **second terminal** for the server:

```
ollama serve                 # leave this running (or: brew services start ollama)
```

Back in the first terminal:

```
ollama pull qwen3:8b         # about 5.2 GB, one time
ollama list                  # should list qwen3:8b
curl http://127.0.0.1:11434/api/tags    # should print JSON that mentions qwen3:8b
```

If the Ollama desktop app is already running, it already provides the server: do not run `ollama serve` as well.

---

4. Test it

4.1 Everything without Ollama

```
python -m pytest -q -m "not live"     # 76 Python tests
npx hardhat test                      # 18 smart contract tests
python scripts/run_demo.py            # full demo with the scripted model
```

The demo starts a local blockchain, runs the agent, anchors its log, then tampers with the log in four ways and shows the verifier catching each one. Expected ending:

```
verify untouched log: INTACT
verify after editing entry 3: TAMPERING DETECTED
verify after consistent rewrite (only the chain can catch this): TAMPERING DETECTED
verify after deleting the last 2 entries: TAMPERING DETECTED
verify after restoring the original: INTACT
```

4.2 With the local Ollama model

Make sure the Ollama server is running and `qwen3:8b` is listed (Step 7). Then:

```
python -m pytest -q -m live -rs       # 3 live tests: local chain, demo, real model
python scripts/run_demo.py --llm ollama
```

The first request is slow while the model loads into memory (it can take a minute); later runs are quick. The real model's wording may differ between runs. The tool calls (`calculator`, `word_count`, `lookup_capital`) and the INTACT / TAMPERING DETECTED results should match.

4.3 Try to break it yourself

```
python scripts/run_demo.py --llm ollama                # creates a log
python scripts/verify_log.py research/experiments/agent_logs/ollama_run.jsonl
```

Expected: `INTACT: 12 entries, hash chain verifies`. Now edit a copy:

```
cp research/experiments/agent_logs/ollama_run.jsonl /tmp/edited.jsonl
sed -i '' 's/Tokyo/Kyoto/' /tmp/edited.jsonl
python scripts/verify_log.py /tmp/edited.jsonl         # prints TAMPERING DETECTED, exit cod
e 1
rm /tmp/edited.jsonl
```

(`sed -i ''` is the macOS form; on Linux use `sed -i`.) Try deleting a line or swapping two lines as well. `verify_log.py` checks only the hash chain. The harder attack, a consistent rewrite, is caught only against the blockchain roots, which the full demo shows.

4.4 More detail

docs/TESTING.md explains what each test file proves and how to read the results.

---

5. Configure

What	Where	Default	How to change
Local model	DEFAULT_MODEL in src/app/ollama_llm.py	qwen3:8b	ollama pull <model>, then change the constant. The model must support tool calling.
Ollama address	DEFAULT_URL in src/app/ollama_llm.py	http://127.0.0.1:11434	Change it if the server runs elsewhere. If you set OLLAMA_HOST for the server, set this to match.
Model randomness	temperature and seed arguments of make_ollama_llm	0.0 and 42	Pass other values where make_ollama_llm is called in src/app/run_demo.py.
Task given to the agent	TASK in src/app/run_demo.py	arithmetic, word count, capital city	Edit the text. The scripted model only understands those three kinds of request.
Entries per anchored batch	BATCH_SIZE in src/app/run_demo.py	5	Larger batches cost less gas but leave more entries unprotected until the next anchor.
Agent step limit	max_steps argument of run_agent in src/domain/agent.py	10	Pass a different value.
Agent tools	DEFAULT_TOOLS and DEFAULT_TOOL_SPECS in src/domain/agent.py	calculator, word counter, capital lookup	Add a function and its description.
Log file location	research/experiments/agent_logs/<model>_run.jsonl	n/a	Git-ignored; each demo run overwrites it.
Blockchain	hardhat.config.js	Hardhat local network, Solidity 0.8.28	The demo starts its own node on a free port each run.
Claude instead of Ollama	ANTHROPIC_API_KEY environment variable	not set	export ANTHROPIC_API_KEY=..., then python scripts/run_demo.py --llm anthropic. Unit-tested but never run live.

To rebuild figures, PDFs and the Word report, install the extra tools with `pip install -r requirements-docs.txt` (see docs/README.md). This is not needed to run or test the project.

---

6. Troubleshooting

First, collect the facts:

```
python --version && node --version && npx hardhat --version && ollama --version
ollama list
curl http://127.0.0.1:11434/api/tags
python -m pytest -q -rs          # -rs prints why each test was skipped
```

Symptom	Likely cause	Fix
ModuleNotFoundError: No module named 'web3' (or pytest)	The Python environment is not active	source .venv/bin/activate, then pip install -r requirements.txt
command not found: npx or npm	Node.js is not installed	Install Node.js from https://nodejs.org, then redo Steps 3 and 4
Chain tests are skipped, or artifacts is missing	The contract is not compiled	npx hardhat compile
npm warn allow-scripts ... during npm install	npm notice about install scripts of some packages	Harmless here: continue with Step 4. If npx hardhat compile and the tests work, nothing is wrong.
Compile fails with a download error	The Solidity compiler is downloaded on first use and needs internet	Reconnect and run npx hardhat compile again
hardhat node did not start in time	A node could not start (Node version problem, or another process)	Run npx hardhat node by hand to see the real error. If it complains about the Node version, try a Node LTS release.
Ollama tests skipped, or Connection refused on port 11434	The Ollama server is not running	ollama serve in another terminal (or brew services start ollama), then re-check curl http://127.0.0.1:11434/api/tags
Error: listen tcp 127.0.0.1:11434: bind: address already in use	Ollama is already running (often the desktop app)	Do not start a second one; just use it
model 'qwen3:8b' not found	The model is not downloaded	ollama pull qwen3:8b, then ollama list
First Ollama run seems stuck for a minute	The model is loading into memory	Wait; later runs are fast. The request times out after 5 minutes.
Error saying the model does not support tools	The configured model has no tool calling	Use a tool-calling model (see https://ollama.com/library) and update DEFAULT_MODEL
Computer is slow or runs out of memory	qwen3:8b needs roughly 6 to 8 GB of RAM	Close other apps, or choose a smaller tool-calling model
The agent's answer differs between runs, or it calls fewer tools	Small local models are not perfectly repeatable	Run again. Whatever the model did is logged, and the log still verifies.
ANTHROPIC_API_KEY is not set	You used --llm anthropic without a key	Use --llm ollama, or export ANTHROPIC_API_KEY=...
INTACT expected but TAMPERING DETECTED shown	The log file was edited or restored from another run	Re-run the demo, which overwrites the log
pip install fails on a package	The package may not support your Python version yet	Use a Python version the error message accepts, then recreate .venv
PDF or figure scripts fail on fonts	They use macOS system fonts	They are optional; the project itself does not need them

Still stuck? Run `python -m pytest -q -rs` and read the skip and failure reasons first, then see docs/TESTING.md section 8.

Stop and clean up. Stop the Ollama server with Ctrl-C in its terminal, or `brew services stop ollama`. Remove demo logs with `rm -rf research/experiments/agent_logs`. Remove the model with `ollama rm qwen3:8b`.

---

7. Repository map

Path	Purpose
contracts/	The Solidity smart contract (AuditAnchor.sol)
chain-tests/	Hardhat tests for the contract
src/core/	Hashing and canonical JSON
src/domain/	Log entries, hash chain, Merkle tree, agent, verifier (pure functions)
src/app/	Blockchain client, log storage, model adapters, demo (side effects)
tests/	Python tests
scripts/	run_demo.py, verify_log.py, and builders for figures, PDFs and the report
docs/diagrams/	19 UML and flow diagrams
docs/thesis/	Technologies explained, IEEE and extended abstracts
docs/study/	Concept study guide with verified links
docs/anna-university/	Report format guide and checklist
docs/report/	Phase I report (Markdown source, Word file, PDF preview)
docs/ (others)	Proposal, literature notes, methodology, notes
assets/figures/	Figures used in the report
research/	Experiments and run logs
.claude/	Project skills and subagents

Full documentation index: docs/README.md. Good starting points: technologies explained, testing guide, diagrams.

---

8. Status

Phase I (third semester). A working prototype: agent, audit log, Merkle batching, smart contract, verifier, model adapters. 76 offline and 3 live Python tests and 18 contract tests pass. Detection of seven kinds of tampering was checked over 300 random logs each, with no misses and no false alarms.

Phase II (next semester). Measure gas against batch size, logging overhead per agent step and verification scaling, then a dashboard. These are planned in docs/methodology/evaluation-plan.md and **not yet done**. The Claude adapter is unit-tested but has never been run against the live service.

Limits. Prototype on a local development chain, not audited. A compromised logger could write false entries before they are anchored. After a consistent rewrite, only the affected batch, not the exact entry, can be located.